

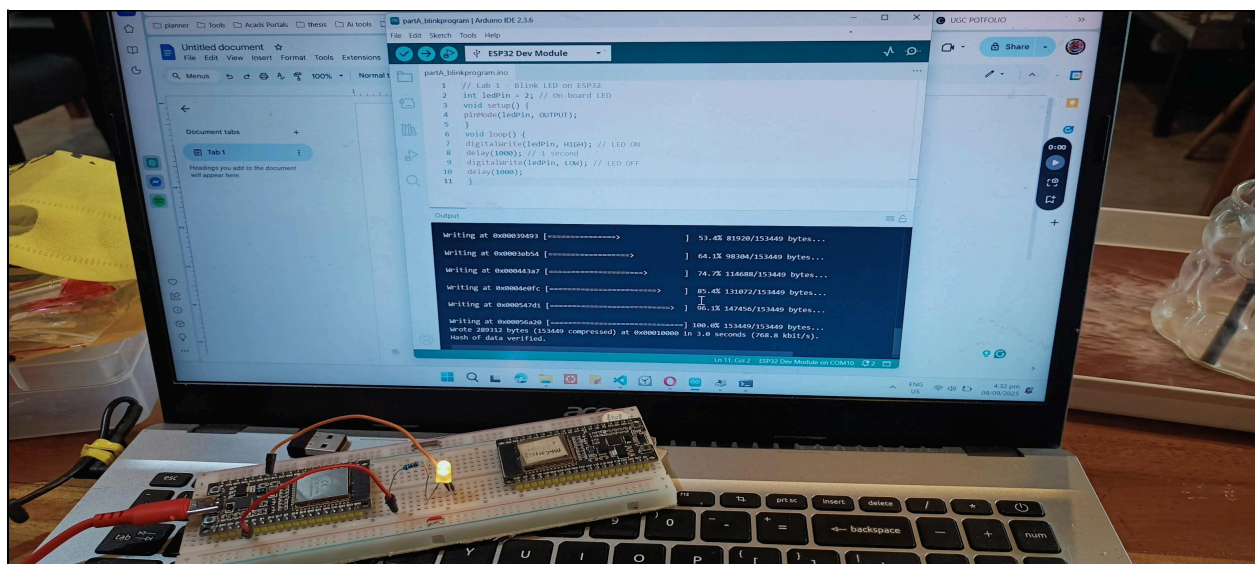
Lab Activity No. 1 ESP32 ↔ Python Two-Way Communication

Procedure (Part A - Blink Program on ESP32)

```
partA_blinkprogram.ino
1 // Lab 1 - Blink + Serial Communication
2 int ledPin = 2;
3
4 void setup() {
5   pinMode(ledPin, OUTPUT);
6   Serial.begin(115200); // Initialize serial communication
7 }
8
9 void loop() {
10  digitalWrite(ledPin, HIGH);
11  Serial.println("Recieved: LED ON"); // Send message
12  delay(1000);
13
14  digitalWrite(ledPin, LOW);
15  Serial.println("Recieved: LED OFF"); // Send message
16  delay(1000);
17 }
```

Output

```
Writing at 0x0003edee [=====>] 60.4% 98304/162829 bytes...
Writing at 0x00044506 [=====>] 70.4% 114688/162829 bytes...
Writing at 0x0004b9dd [=====>] 80.5% 131072/162829 bytes...
Writing at 0x000549c2 [=====>] 90.6% 147456/162829 bytes...
```

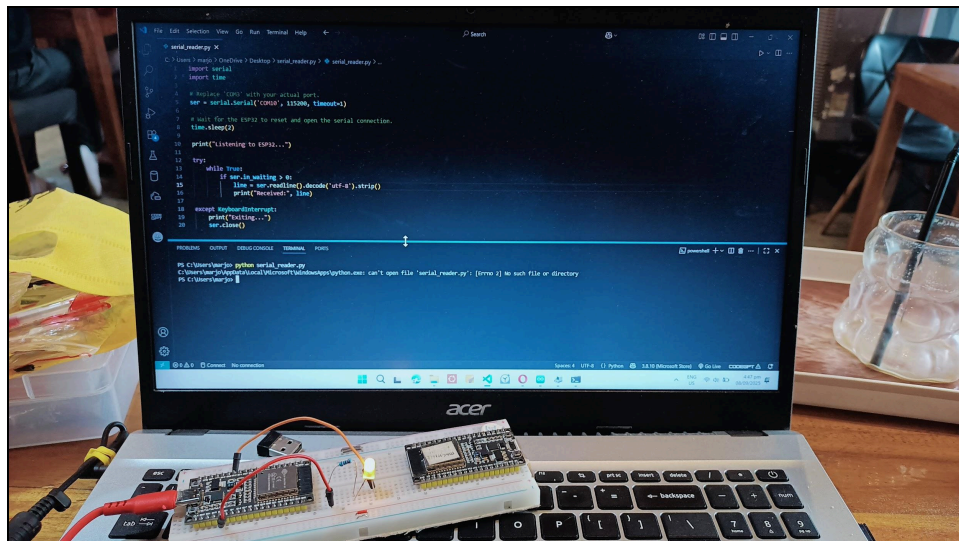


Procedure (Part B - Serial Communication with Python)

```
partB_SerialCommunicationwithPython.ino
1  // Lab 1 - Blink + Serial Communication
2  int ledPin = 2;
3
4  void setup() {
5      pinMode(ledPin, OUTPUT);
6      Serial.begin(115200); // Initialize serial communication
7  }
8
9  void loop() {
10     digitalWrite(ledPin, HIGH);
11     Serial.println("LED ON"); // Send message
12     delay(1000);
13
14     digitalWrite(ledPin, LOW);
15     Serial.println("LED OFF"); // Send message
16     delay(1000);
17 }
```

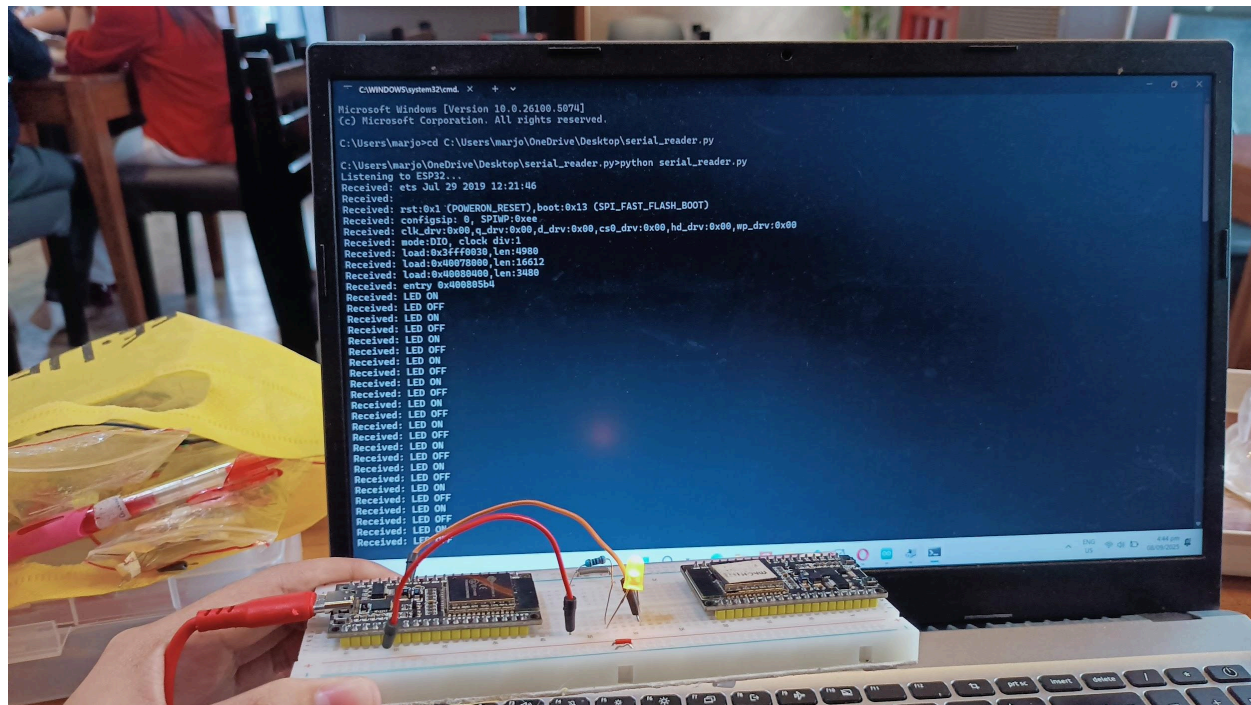
Python Script (serial_reader.py):

```
serial_reader.py X
C: > Users > marjo > OneDrive > Desktop > serial_reader.py > serial_reader.py > ...
1  import serial
2  import time
3
4  # Replace 'COM3' with your actual port.
5  ser = serial.Serial('COM10', 115200, timeout=1)
6
7  # Wait for the ESP32 to reset and open the serial connection.
8  time.sleep(2)
9
10 print("Listening to ESP32...")
11
12 try:
13     while True:
14         if ser.in_waiting > 0:
15             line = ser.readline().decode('utf-8').strip()
16             print("Received:", line)
17
18 except KeyboardInterrupt:
19     print("Exiting...")
20     ser.close()
```



OUTPUT:

[illegible]



Assessment Questions

1. What is the purpose of `Serial.begin(115200)` in the ESP32 sketch?

ANSWER: `Serial.begin(115200)` initializes the UART (Universal Asynchronous Receiver-Transmitter) module on the ESP32. The number 115200 represents the baud rate, which is the speed at which data is transmitted and received in bits per second. By setting this baud rate, you are establishing a communication protocol. Both the ESP32 and the Python script must use the same baud rate to correctly interpret the stream of bits as meaningful characters. This command effectively "opens" a communication channel over the USB port, allowing the ESP32 to send serial data to the connected computer.

2. How does the Python script differentiate between multiple incoming lines?

ANSWER: Differentiating Between Incoming Lines

The Python script differentiates between multiple incoming lines by reading data one line at a time until a line break character is detected.

The `Serial.println()` command in the ESP32 code is crucial here. The `ln` stands for "line," so this function sends a string followed by a newline character (`\n`) and a carriage return (`\r`).

The Python script's `ser.readline()` function is specifically designed to read from the serial buffer until it encounters this newline character (`\n`). This mechanism ensures that each call to `readline()` returns a complete, distinct message, such as "LED ON" or "LED OFF," without mixing them.

3. Modify the ESP32 code to also print the current `millis()` value. What does this Represent?

ANSWER: The `millis()` function returns the number of milliseconds that have passed since the ESP32 board was powered on or reset. This value is an unsigned long integer, continuously increasing until it overflows and wraps back to zero after approximately 50 days. It represents the elapsed time and is fundamentally used for non-blocking timing operations, allowing the microcontroller to perform tasks without relying on `delay()`.

4. Suggest one IoT scenario where Python–ESP32 serial communication can be Useful.

ANSWER: IoT Scenario for Python–ESP32 Serial Communication

A practical IoT scenario where this type of serial communication is useful is a smart hydroponics system.

ESP32 Function: The ESP32 is placed in the plant growing environment. It uses sensors to read real-time data such as soil moisture, temperature, and humidity.

Serial Communication: The ESP32 sends this sensor data to a computer via the serial port.

Python Function: A Python script on the computer acts as a central control and monitoring hub. It reads the incoming sensor data, logs it to a file, and can even visualize it on a simple dashboard.

Two-Way Control: The Python script can analyze the data (e.g., if soil moisture drops below a threshold) and send commands back to the ESP32, such as "PUMP ON" or "PUMP OFF."

This setup demonstrates a fundamental concept of IoT: a low-power, dedicated device (ESP32) handles physical interactions with the environment, while a more powerful computer (running Python) handles data processing, analysis, and higher-level control decisions.